

Take Home: Memory Persistence & Recall for a Conversational Agent

Background

Modern agents maintain persistent memory across sessions. They remember project conventions, stated user preferences, past decisions, and working style, so that a user starting a new conversation doesn't have to re-explain themselves. The agent picks up where it left off.

In this assignment you'll build a small version of that system.

The Problem

Build a conversational agent that gets better at working with a given user over time by remembering what matters from previous conversations and using that context in new ones.

The easy version of this problem is "store everything, retrieve by similarity." That version doesn't work: the memory store fills with noise, retrieval drowns the signal, the assistant grows more confused over time, not less. **The hard and interesting part of this problem is deciding what not to store, when to forget, and how to recover when a memory turns out to be wrong or stale.** A system that remembers everything is useless; a system that forgets the wrong things is worse than no memory at all.

This is an open design problem. Memory has to work across sessions, and it has to work inside long-running conversations. The shape of the solution, what's stored, how, and when it's used, is yours to define.

At a minimum, your submission must:

- Be a working conversational agent that calls a real LLM.
- Survive across sessions - restarting the process, or starting a new conversation, does not erase what was learned.
- Demonstrate end to end that something established in one session meaningfully shapes the agent's behavior in a later session.
- Stay fast as memory grows. End-to-end latency from user input to the first token of the assistant's response must not regress meaningfully as the memory store grows. Specifically: the p50 first-token latency at turn 1,000 (~1,000 stored memories) must be within 200ms of the p50 at turn 1 (cold store).

Document your design choices and the reasoning behind them in the README. We're at least as interested in *why* as in *what*.

Technical Requirements

- Language: your choice. Python or TypeScript preferred.
- Runs locally with clear setup instructions.
- Tests for the core memory logic (capture, persistence, retrieval).
- A README covering: design decisions, tradeoffs, what you'd build next, and time spent.

Constraints

- **No agent frameworks.** Do not use LangChain, LlamaIndex, LangGraph, CrewAI, AutoGen, Semantic Kernel, Haystack, or similar. Call the LLM API directly.
- **No off-the-shelf memory systems.** Do not use mem0, Letta/MemGPT, Zep, MotorHead, or other libraries that implement the memory layer for you. The whole point of this assignment is to see how *you* design it.
- Standard utility libraries are fine - HTTP clients, the official SDK for your chosen LLM provider, test frameworks, embedding models, vector stores (if you use them), database drivers, etc. If you're unsure whether something crosses the line, err on the side of building it yourself, or ask.

Stretch Goals (Optional)

Pick one or two if you have time. We use these to see how you approach harder problems, not as a checklist.

- **Relevance-based retrieval:** when memory grows large, retrieve only the memories most relevant to the current conversation (embeddings + similarity, LLM-based filtering, or another approach).
- **Memory editing:** let the user inspect, edit, or delete memories ("forget that I prefer tabs").
- **Conflict resolution:** detect contradictory memories and reconcile them sensibly.
- **Memory categories:** typed memory (preferences, decisions, facts) with different retention or surfacing rules.
- **Safety:** avoid storing sensitive content (credentials, PII) even when the user shares it.

If you blow your latency budget, fixing it is more valuable than any stretch goal.

Deliverables

A git repository (public, or shared privately with us) containing:

- Source code.
- README with setup instructions, design notes, tradeoffs, and time spent.
- Tests covering the memory logic.
- A demo - transcript, short recording, or a runnable script — that shows memory persisting and being recalled across sessions.

Evaluation Criteria

Area	What we're looking for
Design	Thoughtful choices about what to remember, how to store it, when to surface it.
Judgment	The hardest part is deciding what's worth remembering. Show your reasoning.
Code quality	Clean, readable, idiomatic.
Tradeoffs	A clear articulation of what you chose <i>not</i> to build, and why.
Demonstration	We can run it and watch the memory system actually work.

We are explicitly **not** evaluating UI polish, breadth of features, or use of any specific framework or model.

Time Expectation

Hard limit: 6 hours. Please do not spend more. If you run out of time, document what you'd do next in the README — we value clear thinking over completeness, and we'd rather see an honest "here's where I stopped and why" than a polished submission that took twice as long.

Submission

Send the repo link plus a short note (a few sentences) about what you're most proud of and what you'd change with more time.

If anything is unclear before you start, email us. We'd rather clarify upfront than have you guess.